Use Case Diagram

- Behavioral Diagram
- Use case diagrams model the functionality of a system using actors and use cases.
- A use case diagram is a graphic depiction of the interactions among the elements of a system.
- Use case diagrams are used to depict the context of the system to be built and the functionality provided by that system.
- They show what the outside world wants the system to do.

Why make use case diagrams?

- Use case diagrams are valuable for visualizing the functional requirements of a system that will translate into design choices and development priorities.
- Use case diagrams are used to gather the requirements of a system including internal and external influences. They help to identify the external and internal factors influencing system.
- Use case diagrams can be a good communication tool for stakeholders. The drawings attempt to mimic the real world and provide a view for the stakeholder to understand how the system is going to be designed.
- They help to show the interaction and relationship among the use cases and requirements.

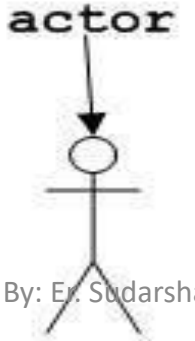
Major Elements of Use Case Diagram

1. Actor

- An actor can be defined as some internal or external entity that interacts with the system.
- Actors are not only roles played by people, but organizations, software, and machines.
- An Actor models a type of role played by an entity that interacts with the subject (e.g., by exchanging signals and data), but which is external to the subject.
- Basically there are two types of actors in a system:

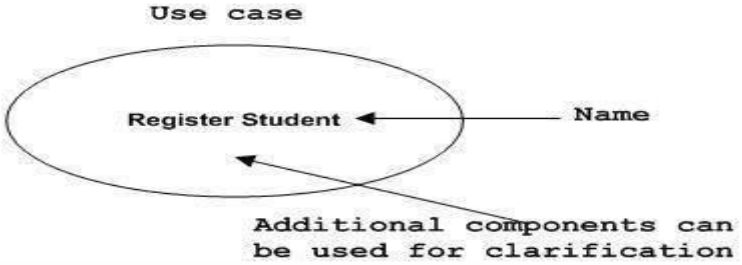
a. Primary Actor: Actor using the system to achieve a goal. Initiates the use of the system.

b. Secondary Actor: Actor that the system needs assistance from to achieve primary actor's goal.



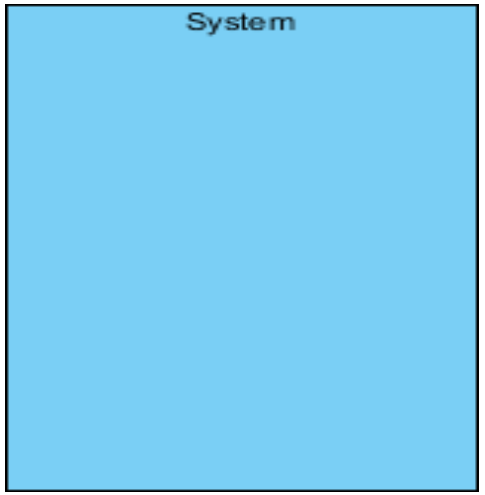
2. Use Cases

- Use cases represent what the actors want your system to do for them.
- A good way of identifying use cases is to recognize what the actor needs from the system or what the actor can do on the system.
- The use case is made up of a set of possible sequences of interactions between systems and users in a particular environment and related to a particular goal.
- Use case **start with verb** and reinforces action that take place.
- In a banking system, a customer will need to open accounts, deposit and withdraw funds, request check books and similar functions. So all of these can be considered as use cases.
- Top level use cases should always provide a complete function required by an actor. You can **extend** or **include** use cases depending on the complexity of the system.



3. System Boundary

- To represent the system a rectangle or rectangular box is used.
- We can use system boundary to group logically related elements.

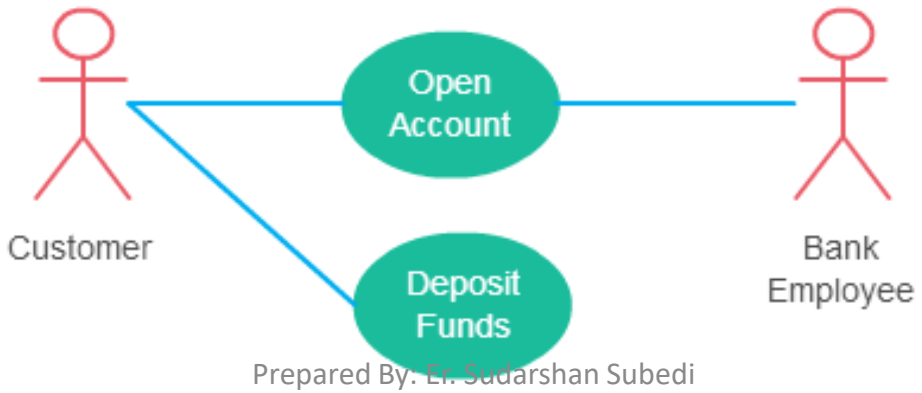


Relationships in use case diagram

1. Association Between Actor and Use Case

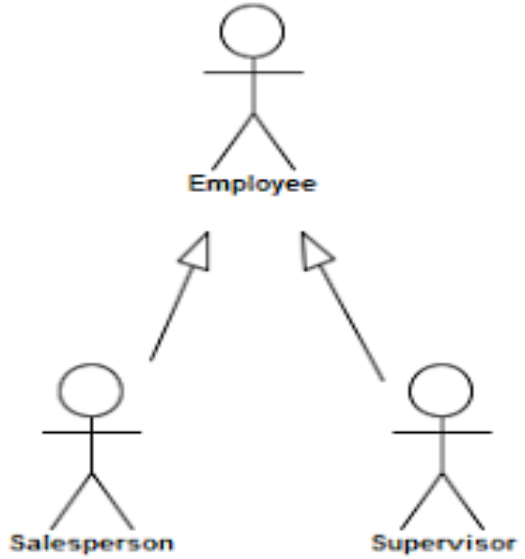
- An association relationship is a structural relationship between two model elements that shows that objects of one classifier (actor, use case, class, interface, node, or component) connect and can navigate to objects of another classifier.
- Association is represented by a solid line.
- Things to remember while using association:

- An actor must be associated with at least one use case.
- An actor can be associated with multiple use cases.
- Multiple actors can be associated with a single use case.



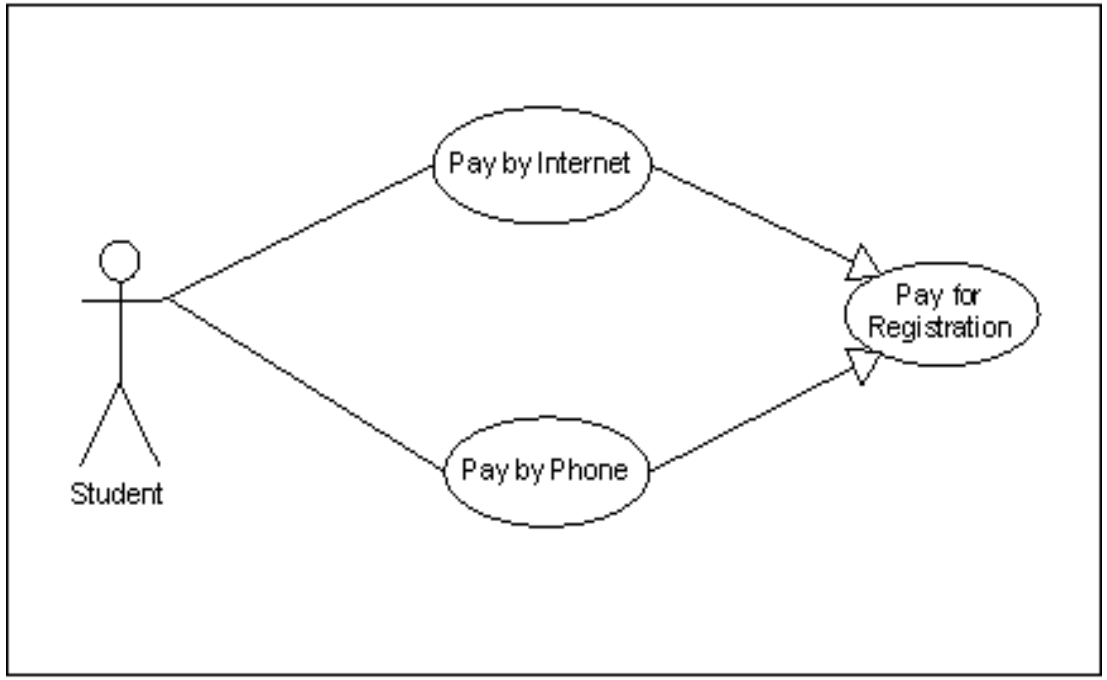
2. Generalization of an actor

- Generalization is communication between an actor and a use case is represented by a solid line.
- Actor generalization refers to the relationship which can exist between two actors and which shows that one actor (descendant) inherits the role and properties of another actor (ancestor).
- The generalization relationship also implies that the descendant actor can use all the use cases that have been defined for its ancestor. i.e. the descendant inherits all the use cases of the ancestor.



3. Generalization of Use Case

- In generalization of use case, the behavior of the ancestor use case is inherited by the descendant use case.
- We can use the generalization relationship when we find two or more use cases which have common behavior/logic.



4. Include Relationship

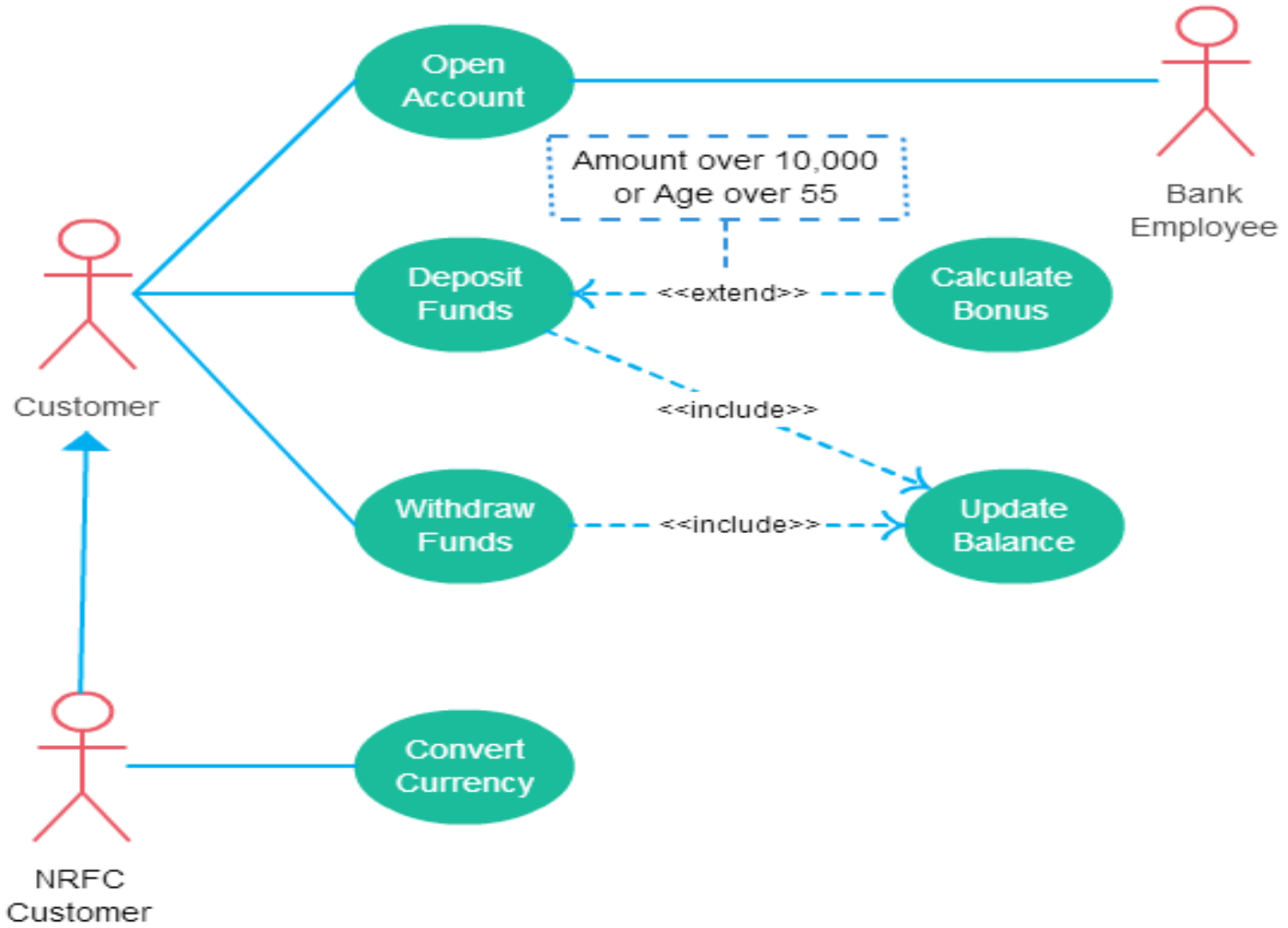
- Include is a relationship between two use cases which is used to show that behavior of the included use case is inserted into the behavior of the including (the base) use case.
- Use case include is a directed relationship between two use cases which is used to show that behavior of the included use case is inserted into the behavior of the including (the base) use case.
- Every time the base use case is executed, the included use case is executed as well.
- To show include relationship, an arrow line from base to included use case.
- The **include** relationship could be used to:

a. simplify large use case by splitting it into several use cases

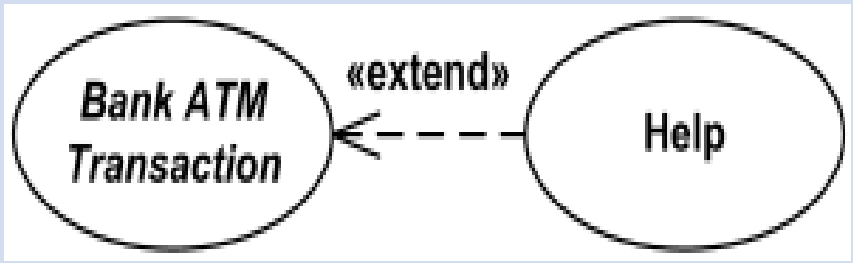
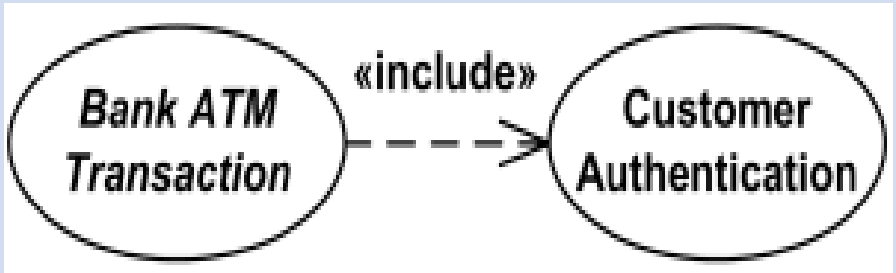
b. extract common parts of the behaviors of two or more use cases

5. Extend Relationship

- Extend is a directed relationship that specifies how and when the behavior defined in extending use case can be inserted into the behavior defined in the extended(base) use case.
- Extending use case typically defines **optional behavior** that is not necessarily meaningful by itself.
- There are some functions that are triggered optionally. In such cases, we can use the extend relationship.
- When the base use case is executed, the extending use case is executed only after meeting certain criteria.
- To show extend relationship, an arrow line from extending to base use case is used.


Open Account
Bank Employee
Amount over 10,000 or Age over 55
Deposit Funds
<<extend>>
Calculate Bonus
Customer
<<include>>
Withdraw Funds
<<include>>
Update Balance
Convert Currency
NRFC Customer

Extend vs Include Relationship

Extend	Include
Extend is a directed relationship that specifies how and when the behavior defined in extending use case can be inserted into the behavior defined in the extended(base) use case.	Include is a relationship between two use cases which is used to show that behavior of the included use case is inserted into the behavior of the including (the base) use case.
In extend relationship, base use case is complete on its own.	In include relationship, included use case is required.
Extending use case is optional.	Included use case is required.
To show extend relationship, an arrow line from extending to base use case is used.	To show include relationship, an arrow line from base to included use case.
	

How to draw a use case diagram?

1. Identifying Actor: Actors are external entities that interact with your system. It can be a person, another system or an organization.

Such actors are identified in this phase. In a banking system, the most obvious actor is the customer. Other actors can be bank employee or cashier depending on the role.

Primary Actor: Actor using the system to achieve a goal. Initiates the use of the system.

Secondary Actor: Actor that the system needs assistance from to achieve primary actor's goal.

2. Identifying Use Case: In this phase the use cases needed are to be identified. A good way to do this is to identify what the actor needs from the system or what the actor can do on the system.

Use case **start with verb** and reinforces action that take place.

In a banking system, a customer will need to open accounts, deposit and withdraw funds, request check books and similar functions. So all of these can be considered as use cases.

Top level use cases should always provide a complete function required by an actor. You can extend or include use cases depending on the complexity of the system.

3. Look for Common Functionality to use include relationship: Include is a relationship between two use cases which is used to show that behavior of the included use case (the addition) is inserted into the behavior of the including (the base) use case.

It is a relationship in which one use case includes the functionality of another use case.

In case of include relationship, generally it exists between a base use case and included use case. Every time the base use case is executed, the included use case is executed as well.

To show relationship: Arrow line from base to included use case.

4. Is it Possible to Generalize Actors and Use Cases: We need to check whether the actors and use cases can be generalized or not. We can generalize the actor and use cases to show the inheritance of functions.

One of the best examples of this is "Make Payment" use case in a payment system. You can further generalize it to "Pay by Credit Card", "Pay by Cash", "Pay by Check" etc. All of them have the attributes and the functionality of a payment with special scenarios unique to them.

5. Optional Functions or Additional Functions: There are some functions that are triggered optionally. In such cases, we can use the extend relationship.

These sorts of function only gets triggered when specified condition is matched.

In UML modeling, we can use an **extend relationship** to specify that one use case extends the behavior of another. This type of relationship reveals details about a system or application that are typically hidden in a use case.

An extend relationship has a base use case and extending use case. When the base use case is executed, the extending use case is executed only after meeting certain criteria.

To show relationship: Arrow line from extending to base use case.

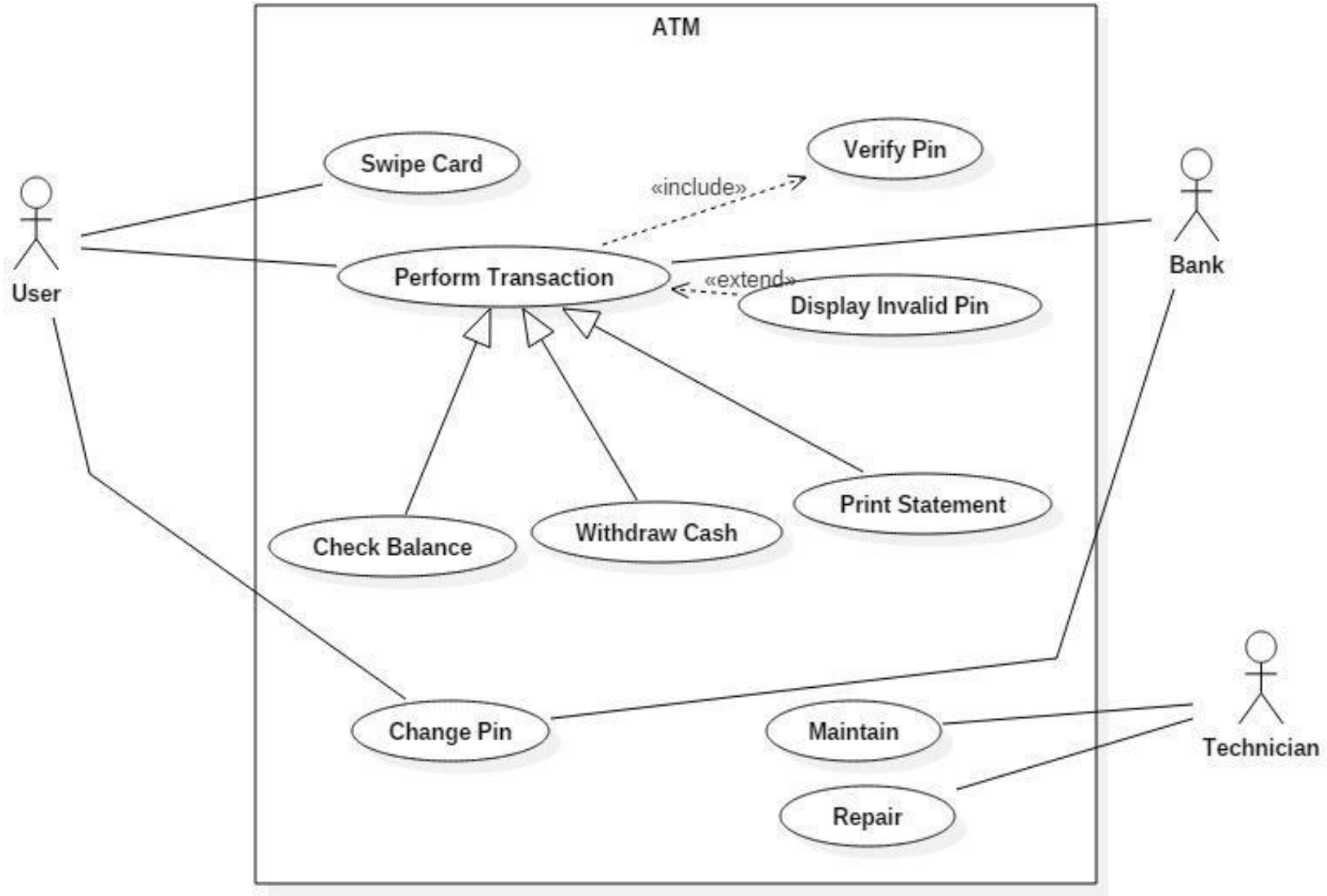


Figure: Use Case Diagram For ATM System

Example

MathTrainer aids in perfecting the mental arithmetic of elementary school students. MathTrainer poses each student ten random arithmetical exercises, which should be solved as fast and correct as possible. From the responses, scores are collected which can be viewed by the users of MathTrainer.

Teachers can define types of exercises by determining numerical ranges and allowed mathematical operations. They can also delete types which were defined by themselves. Students are assigned to their teacher and can request exercises for an exercise type of their teacher. New teachers and students are able to apply as new MathTrainer users themselves by specifying username and password – this is done in the context of the usual user identification. The password can be changed anytime. Teachers can delete the students which are assigned to them. During the realization of a test (ten exercises) the time needed for each exercise is stopped.

However, the scores are based on the cumulative time.